

Build a Complete RAG Application

Python + FastAPI + OpenAI + ChromaDB

Full student notes and runnable scripts

PDF, DOCX and TXT ingestion | Chunking | Embeddings | Vector search | Grounded answers | Sources | Memory | Filtering | Deletion

Learning outcome: Build a simple Retrieval-Augmented Generation application from scratch and test every API using Swagger UI.

1. What is RAG?

RAG means Retrieval-Augmented Generation. It first retrieves relevant information from our documents and then gives that information to a Large Language Model (LLM) to generate a grounded answer.

Simple formula: User question + Retrieved document context + Instructions = Grounded answer

Why use RAG?

- The LLM can answer from private or newly uploaded documents.
- Relevant chunks are selected instead of sending an entire document.
- Sources such as filename and page number can be returned.
- A strict prompt reduces unsupported answers and hallucinations.
- Documents can be changed without retraining the LLM.

Application workflow

Upload document -> Extract text -> Split into chunks -> Generate embeddings -> Store in ChromaDB -> Embed question -> Similarity search -> Retrieve top 3 chunks -> Send context to LLM -> Return answer and sources.

Main components

Component	Purpose
FastAPI	Creates upload, ask, list, memory and delete APIs
PyPDF / python-docx	Extracts text from supported files
OpenAI embedding model	Converts text and questions into vectors
ChromaDB	Stores vectors, text chunks and metadata
OpenAI chat model	Produces an answer using retrieved context

2. Project Structure

```
simple-rag-app/  
|-- main.py  
|-- requirements.txt  
|-- .env  
|-- uploads/          # Created automatically  
-- chroma_database/    # Created automatically
```

Prerequisites

- Python 3.10 or newer
- An OpenAI API key
- Internet connection for OpenAI API calls
- VS Code, PyCharm or another editor

Create and activate virtual environment - Windows

```
mkdir simple-rag-app  
cd simple-rag-app  
py -m venv venv  
venv\Scripts\activate
```

Create and activate virtual environment - macOS/Linux

```
mkdir simple-rag-app
cd simple-rag-app
python3 -m venv venv
source venv/bin/activate
```

3. requirements.txt

```
fastapi
uvicorn
python-multipart
chromadb
openai
pypdf
python-docx
python-dotenv
```

Install packages

```
pip install -r requirements.txt
```

Package purpose

- python-multipart receives uploaded files and form fields.
- chromadb stores vectors, chunks and metadata locally.
- pypdf reads normal text-based PDF files.
- python-docx reads DOCX paragraphs.
- python-dotenv loads secrets and model names from .env.

4. Environment Configuration

Create a file named **.env** in the project root. Never share a real API key or commit it to Git.

```
OPENAI_API_KEY=your_openai_api_key_here
OPENAI_CHAT_MODEL=gpt-4.1-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

Replace **your_openai_api_key_here** with a key created in your own OpenAI account. Model availability can vary by account; update the model names if required.

5. Complete main.py

The following single script implements document upload, extraction, chunking, embeddings, persistent vector storage, question retrieval, grounded generation, sources, memory, filtering, listing and deletion.

```
import os
import uuid
from pathlib import Path

import chromadb
from docx import Document
from dotenv import load_dotenv
from fastapi import FastAPI, File, Form, UploadFile
from openai import OpenAI
from pydantic import BaseModel
from pypdf import PdfReader

load_dotenv()

app = FastAPI(title="Simple Python RAG Application")
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

CHAT_MODEL = os.getenv("OPENAI_CHAT_MODEL", "gpt-4.1-mini")
EMBEDDING_MODEL = os.getenv(
    "OPENAI_EMBEDDING_MODEL", "text-embedding-3-small"
)

UPLOAD_FOLDER = Path("uploads")
UPLOAD_FOLDER.mkdir(exist_ok=True)

chroma_client = chromadb.PersistentClient(path="./chroma_database")
collection = chroma_client.get_or_create_collection(name="rag_documents")
```

```
# Simple memory. Production applications should use a database or Redis.
conversation_memory = {}
```

```
class QuestionRequest(BaseModel):
    question: str
    session_id: str = "default-session"
    filename: str | None = None
    category: str | None = None
```

```
def extract_pdf(file_path):
    reader = PdfReader(file_path)
    pages = []
    for page_number, page in enumerate(reader.pages, start=1):
        pages.append({
            "text": page.extract_text() or "",
            "page": page_number
        })
    return pages
```

```
def extract_docx(file_path):
    document = Document(file_path)
    text = "\n".join(p.text for p in document.paragraphs)
    return [{"text": text, "page": 1}]
```

```
def extract_txt(file_path):
    text = Path(file_path).read_text(encoding="utf-8", errors="ignore")
    return [{"text": text, "page": 1}]
```

```
def extract_text(file_path):
    extension = Path(file_path).suffix.lower()
    if extension == ".pdf":
        return extract_pdf(file_path)
    if extension == ".docx":
        return extract_docx(file_path)
    if extension == ".txt":
        return extract_txt(file_path)
    return []
```

```
def split_text(text, chunk_size=900, overlap=150):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
            start += chunk_size - overlap
    return chunks
```

```
def generate_embedding(text):
    response = openai_client.embeddings.create(
        model=EMBEDDING_MODEL,
        input=text
    )
    return response.data[0].embedding
```

```
@app.get("/")
def home():
    return {"message": "Simple RAG application is running"}
```

```
@app.post("/upload")
async def upload_document(
    file: UploadFile = File(...),
    category: str = Form("general")
):
    document_id = str(uuid.uuid4())
    file_path = UPLOAD_FOLDER / file.filename
    file_path.write_bytes(await file.read())
    pages = extract_text(file_path)

    if not pages:
        return {"message": "Only PDF, DOCX and TXT files are supported"}

    chunk_count = 0
    for page_data in pages:
        chunks = split_text(page_data["text"])
        for chunk in chunks:
            chunk_id = str(uuid.uuid4())
            embedding = generate_embedding(chunk)
            collection.add(
```

```

        ids=[chunk_id],
        documents=[chunk],
        embeddings=[embedding],
        metadatas=[{
            "document_id": document_id,
            "filename": file.filename,
            "category": category,
            "page": page_data["page"]
        }]
    )
    chunk_count += 1

return {
    "message": "Document uploaded successfully",
    "document_id": document_id,
    "filename": file.filename,
    "category": category,
    "chunks_created": chunk_count
}

@app.post("/ask")
def ask_question(request: QuestionRequest):
    question_embedding = generate_embedding(request.question)

    filters = []
    if request.filename:
        filters.append({"filename": request.filename})
    if request.category:
        filters.append({"category": request.category})

    where_filter = None
    if len(filters) == 1:
        where_filter = filters[0]
    elif len(filters) > 1:
        where_filter = {"$and": filters}

    query_parameters = {
        "query_embeddings": [question_embedding],
        "n_results": 3,
        "include": ["documents", "metadatas", "distances"]
    }
    if where_filter:
        query_parameters["where"] = where_filter

    results = collection.query(**query_parameters)
    retrieved_chunks = results["documents"][0]
    metadata_list = results["metadatas"][0]
    distances = results["distances"][0]

    if not retrieved_chunks:
        return {
            "answer": "The answer is not available in the uploaded documents.",
            "sources": []
        }

    # This is a demonstration threshold. Tune it with evaluation data.
    if distances and distances[0] > 1.5:
        return {
            "answer": "The answer is not available in the uploaded documents.",
            "sources": []
        }

    context = "\n\n".join(retrieved_chunks)
    history = conversation_memory.get(request.session_id, [])
    history_text = "\n\n".join(
        f"{item['role']}: {item['content']}" for item in history[-6:]
    )

    prompt = f"""
You are a document-based question-answering assistant.
Answer the question using ONLY the provided context.

Rules:
1. Do not use outside knowledge.
2. Do not create or assume information.
3. If the answer is unavailable, respond exactly:
   The answer is not available in the uploaded documents.
4. Keep the answer clear and concise.

Conversation history:
{history_text}

Context:
{context}

Question:
{request.question}
"""

```

```

response = openai_client.responses.create(
    model=CHAT_MODEL,
    input=prompt
)
answer = response.output_text

conversation_memory.setdefault(request.session_id, [])
conversation_memory[request.session_id].extend([
    {"role": "user", "content": request.question},
    {"role": "assistant", "content": answer}
])

sources = []
for metadata in metadata_list:
    source = {
        "filename": metadata["filename"],
        "page": metadata["page"],
        "category": metadata["category"]
    }
    if source not in sources:
        sources.append(source)

return {
    "question": request.question,
    "answer": answer,
    "sources": sources
}

@app.get("/memory/{session_id}")
def get_memory(session_id: str):
    return {
        "session_id": session_id,
        "messages": conversation_memory.get(session_id, [])
    }

@app.delete("/memory/{session_id}")
def clear_memory(session_id: str):
    conversation_memory.pop(session_id, None)
    return {"message": "Conversation memory cleared"}

@app.get("/documents")
def get_documents():
    data = collection.get(include=["metadatas"])
    documents = {}
    for metadata in data["metadatas"]:
        document_id = metadata["document_id"]
        documents[document_id] = {
            "document_id": document_id,
            "filename": metadata["filename"],
            "category": metadata["category"]
        }
    return list(documents.values())

@app.delete("/documents/{document_id}")
def delete_document(document_id: str):
    document_data = collection.get(
        where={"document_id": document_id},
        include=["metadatas"]
    )
    if not document_data["ids"]:
        return {"message": "Document not found"}

    filename = document_data["metadatas"][0]["filename"]
    collection.delete(where={"document_id": document_id})

    file_path = UPLOAD_FOLDER / filename
    if file_path.exists():
        file_path.unlink()

    return {
        "message": "Document and embeddings deleted successfully",
        "document_id": document_id
    }

```

6. Run the Application

```
uvicorn main:app --reload
```

Open Swagger UI in the browser: <http://127.0.0.1:8000/docs>

What happens at startup?

- FastAPI creates the web application.
- The .env values are loaded.
- The OpenAI client is initialized.
- uploads/ is created if missing.
- ChromaDB opens or creates ./chroma_database.
- The rag_documents collection is opened or created.

7. Test the APIs in Swagger

Step 1 - Check application

GET /

Expected response:
{"message": "Simple RAG application is running"}

Step 2 - Upload a document

POST /upload

file: employee_policy.pdf
category: hr

Example response:

```
{
  "message": "Document uploaded successfully",
  "document_id": "generated-uuid",
  "filename": "employee_policy.pdf",
  "category": "hr",
  "chunks_created": 18
}
```

Step 3 - Ask a question

POST /ask

```
{
  "question": "How many casual leaves are available?",
  "session_id": "student-101",
  "filename": "employee_policy.pdf",
  "category": "hr"
}
```

Example answer

```
{
  "question": "How many casual leaves are available?",
  "answer": "Employees receive 12 casual leaves per year.",
  "sources": [
    {
      "filename": "employee_policy.pdf",
      "page": 4,
      "category": "hr"
    }
  ]
}
```

Step 4 - List documents

GET /documents

Step 5 - Read or clear memory

GET /memory/student-101
DELETE /memory/student-101

Step 6 - Delete document and vectors

DELETE /documents/{document_id}

8. How the Code Works

Text extraction

`extract_text()` checks the extension and calls the correct reader. PDF text is retained page by page, allowing the answer to show a real PDF page number. DOCX and TXT are treated as page 1 because these formats do not provide stable PDF-style pages during simple extraction.

Chunking

The script uses a chunk size of 900 characters and an overlap of 150 characters. Overlap repeats the end of one chunk at the beginning of the next, reducing the chance that an important sentence is cut at a boundary.

Embeddings

`generate_embedding()` sends text to the embedding model. The result is a numerical vector representing semantic meaning. Similar meanings should be located closer together in vector space.

Vector storage

Each ChromaDB record contains a unique chunk ID, original text, embedding and metadata: `document_id`, `filename`, `category` and `page`. ChromaDB is persistent because `PersistentClient` uses `./chroma_database`.

Retrieval

The question is also converted into an embedding. `collection.query()` compares it with stored vectors and retrieves the top three semantically similar chunks. Optional `filename` and `category` filters narrow the search.

Grounded generation

The retrieved chunks become context in the prompt. The model is explicitly told to use only the context and to return a fixed fallback message when the answer is unavailable.

Memory

Messages are saved by `session_id` in a Python dictionary. Only the latest six messages are placed in the prompt. This is suitable for a demonstration, but it is erased whenever the application restarts.

9. Mapping the 15 Coding Requirements

No.	Requirement	Implemented by
1	Load PDF	<code>extract_pdf()</code> uses <code>PdfReader</code> .
2	Split text	<code>split_text()</code> creates overlapping chunks.
3	Generate embeddings	<code>generate_embedding()</code> calls the embedding model.
4	Store vectors	<code>collection.add()</code> stores chunks in ChromaDB.
5	Embed question	<code>ask_question()</code> calls <code>generate_embedding(question)</code> .
6	Top 3 search	<code>collection.query(n_results=3)</code> .
7	Pass context to LLM	Retrieved chunks are inserted into prompt.
8	Answer with source	Response contains <code>filename</code> , <code>page</code> and <code>category</code> .
9	Conversation memory	<code>conversation_memory</code> stores messages by session.
10	POST /upload	Accepts file and category.
11	POST /ask	Accepts question, session and filters.

No.	Requirement	Implemented by
12	PDF/DOCX/TXT	extract_text() routes all three formats.
13	Prevent outside answer	Strict prompt plus demonstration distance threshold.
14	Delete vectors	DELETE endpoint removes Chroma records and file.
15	Metadata filtering	where filter supports filename and category.

10. Important Concepts

Chunk size and overlap

Small chunks improve precision but may lose surrounding meaning. Large chunks preserve context but add irrelevant content and token cost. There is no universal best value; evaluate using questions from the real domain.

Top-K

Top-K is the number of chunks retrieved. This project uses K=3. A very small K may miss evidence; a very large K may introduce noise and cost.

Similarity distance

The sample rejects the answer when the best distance is above 1.5. This value is only a classroom starting point. Distance meaning depends on the embedding model and collection configuration, so a production threshold must be calibrated using labelled relevant and irrelevant questions.

Metadata

Metadata is descriptive information stored beside a chunk. It enables filtering, source display and document-level deletion without searching the text itself.

RAG is not model training

RAG retrieves context at question time. Fine-tuning changes model behaviour through training examples. Uploading documents to a vector store does not retrain the LLM.

11. Common Problems and Solutions

Problem	Likely solution
401 invalid_api_key	Check OPENAI_API_KEY in .env, remove spaces and restart Uvicorn.
ModuleNotFoundError	Activate the correct virtual environment and install requirements.
Empty PDF text	The PDF may be scanned. Add OCR such as Tesseract for image-based pages.
No useful answer	Verify extraction, chunk size, model, filters, query and stored metadata.
Chroma dimension error	Use the same embedding model for stored chunks and questions; rebuild incompatible data.
Old code after editing	Stop and restart Uvicorn, or use --reload during development.
Memory disappears	The dictionary is temporary; store messages in MySQL, PostgreSQL or Redis.
Duplicate uploads	Create a file hash before ingestion and reject a previously stored hash.

12. Limitations and Production Improvements

- Validate file type, filename, file size and empty content.

- Sanitize filenames and use a unique server-side storage name.
- Hash files to prevent duplicate ingestion.
- Batch embedding requests to reduce network calls.
- Add authentication and isolate data by user or tenant.
- Store document records and conversation history in a database.
- Use OCR for scanned PDFs and images.
- Add retries, timeouts, logging and structured exception handling.
- Evaluate retrieval with a labelled question-answer dataset.
- Add a reranker, citations linked to excerpts and prompt-injection protection.
- Do not rely on one fixed distance threshold without testing.
- Use background jobs for large document ingestion.

13. Student Coding Assignments

- Add a GET /documents/{document_id} endpoint.
- Reject files other than PDF, DOCX and TXT.
- Add file-size limit of 20 MB.
- Calculate SHA-256 and prevent duplicate uploads.
- Let the user choose top_k between 1 and 10.
- Return the retrieved text excerpts with sources.
- Store conversation memory in MySQL.
- Add OCR support for scanned PDFs.
- Create a simple HTML, CSS and JavaScript frontend.
- Add a category endpoint and filter UI.
- Compare chunk sizes 400, 900 and 1500.
- Replace ChromaDB with FAISS or PGVector.

14. Interview and Viva Questions

1. What problem does RAG solve?
2. How is RAG different from fine-tuning?
3. Why do we split documents into chunks?
4. What is chunk overlap?
5. What is an embedding?
6. Why must documents and questions use the same embedding model?
7. What is cosine similarity or vector distance?
8. What does Top-K mean?

9. What does ChromaDB store?
10. Why is metadata useful?
11. How do filename and category filters work?
12. How can RAG hallucinations be reduced?
13. Why should a similarity threshold be evaluated?
14. How would you prevent duplicate ingestion?
15. How would you support scanned PDFs?
16. How would you delete every chunk of one document?
17. Why is in-memory chat history unsuitable for production?
18. What is hybrid search?
19. What is reranking?
20. How would you evaluate retrieval quality?
21. How would you secure documents for multiple users?
22. What happens if the embedding model changes?
23. How can prompt injection inside a document affect RAG?
24. How would you reduce latency and API cost?
25. When should you use RAG instead of sending the whole document?

15. Quick Revision

Indexing phase: Load -> Extract -> Chunk -> Embed -> Store

Question phase: Embed question -> Search -> Retrieve -> Build context -> Generate -> Cite

Core rule: A RAG answer is only as reliable as its document extraction, chunking, retrieval, grounding instructions and evaluation.

Final checklist

- Virtual environment activated
- Packages installed
- .env contains your own API key
- Application runs without import errors
- Swagger UI opens
- Document uploads and creates chunks
- Question returns an answer and sources
- Filtering works
- Memory can be read and cleared
- Document deletion removes its embeddings